

Print double-sided: diagrams this side, explanations on the back.

Figure 1 consists of six sub-diagrams, labeled (a) through (f), each showing a network of nodes and edges. The nodes are represented by circles, and the edges are represented by lines. The diagrams illustrate the evolution of a network structure, specifically focusing on the concept of chordality. Each diagram shows a central node connected to three peripheral nodes, with additional edges forming a triangle between the peripherals. The labels for each diagram are as follows:

- (a) Original Chordality
A: Chordal
B: Chordal
C: Chordal
- (b) Chordality
A: Chordal
B: Chordal
C: Chordal
- (c) Chordality
A: Chordal
B: Chordal
C: Chordal
- (d) Chordality
A: Chordal
B: Chordal
C: Chordal
- (e) Chordality
A: Chordal
B: Chordal
C: Chordal
- (f) Chordality
A: Chordal
B: Chordal
C: Chordal

$\text{Pivot } p = \mathcal{A}[l] \text{ (first element)}$

 $[42 \ 26 \ 41 \ 56 \ 43 \ 33]$
 $\quad \quad \quad i \quad \quad \quad j$
 1. *i* steps at first number > 42 > 56
 $j \ @ \ index \ 4$
 $\mathcal{A}[j] = 56$

 $[42 \ 26 \ 41 \ 56 \ 43 \ 33]$
 $\quad \quad \quad i \quad \quad \quad j$
 2. *j* steps at first number < 42 > 33 (already there)
 $j \ @ \ index \ 4$
 $\mathcal{A}[j] = 33$

 $[42 \ 26 \ 41 \ 56 \ 43 \ 33]$
 $\quad \quad \quad i \quad \quad \quad j$
 3. $i < j$ ($4 < 6$) $\rightarrow$ swap $\mathcal{A}[i], \mathcal{A}[j]$

 $[42 \ 26 \ 41 \ 33 \ 43 \ 56]$
 $\quad \quad \quad i \quad \quad \quad j$
 4. *i* advances rightward to next number > 42 > 43
 $i \ @ \ index \ 5$
 $\mathcal{A}[i] = 43$

 $[42 \ 26 \ 41 \ 33 \ 43 \ 56]$
 $\quad \quad \quad i \quad \quad \quad j$
 5. *j* advanced leftward to next number < 42 > 33
 $j \ @ \ index \ 4$
 $\mathcal{A}[j] = 33$

 $[42 \ 26 \ 41 \ 33 \ 43 \ 56]$
 $\quad \quad \quad j \quad \quad \quad i$
 6. $i > j$ ($5 > 4$) $\rightarrow$ stop; swap $\mathcal{A}[i], \mathcal{A}[j]$

 $[33 \ 26 \ 41 \ 42 \ 43 \ 56]$
 $\quad \quad \quad j \quad \quad \quad i$
Results after one full partition:
 $(\text{first element})(\text{pivot})$

[31, 10, 24, 1, 12]

Children: 1 (idx 3), 12 (idx 4)
 Larger child: 12 (idx 4) $\rightarrow$ swap 10 and 12

Final Heap:

(a) (b) (c) (d) (e)

Starting with shortest edge
 $W(2) = 1$

Next shortest edge
 $W(3) = 2$

Next shortest
 $W(4) = 3$

Last shortest, final $W(8)$
 $W(5) = 4$
 Total weight is 4
 $4 + 1 + 2 + 2 = 9$

4. Start from any vertex, A
 - greedy $\rightarrow$ choose smallest edge that
 doesn't create an unwanted rule

Valid: [B]

5. Choosing smallest edge: 2
 - creates one false MSF

Valid: [B, D]

6. Rejects, next smallest edge: 3
 - $\rightarrow$ RD [3]
 - AC [3] $\rightarrow$ Choosing this one
 - DE [3]

Valid: [B, D, C]

7. Rejects, next smallest edge: 1
 - $\rightarrow$ CE [1]

Valid: [B, B, C, E]

8. Rejects, next smallest edge: 3
 - $\rightarrow$ ED [3]

Valid: [B, B, C, E, F]

9. Rejects, next smallest edge: 6
 - $\rightarrow$ CF [6]

Valid: [B, B, C, E, F, F]

10. Rejects, next smallest edge: 8
 - $\rightarrow$ EF [8]

Valid: [B, B, C, E, F, F, F]

Compare every pair sizes. In each pair, the larger number is correct game play by 1

Compare every pair sizes. In each pair, the larger number is correct game play by 1			
winner 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1	runner-up 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1	winner 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1	runner-up 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

(Plus number in their respective index position <=max(i))

new ordering: [1, 5, 10, 12, 24, 31]

[illegible]

C_1 Coins have currently setting
 m : current amount
 being adding for
 the next step a

If you are answer
 please note needed
 to make amount a

$f(m) = m$ when C_1 coins of C_1 value $C_1 = 1$
 (1, 1, 1, 1) need a target $a = 10$

1. **Generate** $dp[0]$ value a
 - cost of 0 coins to get the initial state
 - amount we already added
 - C_1 coins: 1 coin jump leads on $f(0)$ then state coins
 - $m = f(0) = 0$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0

2. **Generate** $dp[1]$ value a
 - $m = f(1) = 1$
 - $\text{min}(f[0] + 1, f[1] + 1 = 1)$
 - C_1 coins: 2 coin jump leads on $f(0)$ then state coins
 - $m = f(1) = 1$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1

3. **Generate** $dp[2]$ value a
 - $m = f(2) = 2$
 - $\text{min}(f[0] + 1, f[1] + 1 = 2)$
 - C_1 coins: 3 coin jump leads on $f(0)$ then state coins
 - $m = f(2) = 2$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2

4. **Generate** $dp[3]$ value a
 - $m = f(3) = 3$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 2)$
 - C_1 coins: 4 coin jump leads on $f(0)$ then state coins
 - $m = f(3) = 3$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3

5. **Generate** $dp[4]$ value a
 - $m = f(4) = 4$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2)$
 - C_1 coins: 5 coin jump leads on $f(0)$ then state coins
 - $m = f(4) = 4$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4

6. **Generate** $dp[5]$ value a
 - $m = f(5) = 5$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2, f[4] + 1 = 2)$
 - C_1 coins: 6 coin jump leads on $f(0)$ then state coins
 - $m = f(5) = 5$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4 5

7. **Generate** $dp[6]$ value a
 - $m = f(6) = 6$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2, f[4] + 1 = 2, f[5] + 1 = 2)$
 - C_1 coins: 7 coin jump leads on $f(0)$ then state coins
 - $m = f(6) = 6$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4 5 6

8. **Generate** $dp[7]$ value a
 - $m = f(7) = 7$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2, f[4] + 1 = 2, f[5] + 1 = 2, f[6] + 1 = 2)$
 - C_1 coins: 8 coin jump leads on $f(0)$ then state coins
 - $m = f(7) = 7$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4 5 6 7

9. **Generate** $dp[8]$ value a
 - $m = f(8) = 8$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2, f[4] + 1 = 2, f[5] + 1 = 2, f[6] + 1 = 2, f[7] + 1 = 2)$
 - C_1 coins: 9 coin jump leads on $f(0)$ then state coins
 - $m = f(8) = 8$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4 5 6 7 8

10. **Generate** $dp[9]$ value a
 - $m = f(9) = 9$
 - $\text{min}(f[0] + 1, f[1] + 1 = 3, f[2] + 1 = 3, f[3] + 1 = 2, f[4] + 1 = 2, f[5] + 1 = 2, f[6] + 1 = 2, f[7] + 1 = 2, f[8] + 1 = 2)$
 - C_1 coins: 10 coin jump leads on $f(0)$ then state coins
 - $m = f(9) = 9$

m 0 1 2 3 4 5 6 7 8 9 10
 $f(m)$ 0 1 2 3 4 5 6 7 8 9

[illegible]

1. Pick *earliest* node: *A*
Update costs chart

Unvisited nodes = {B, C, D, E}

A: 0
B: 1
C: 2
D: 3
E: 4

2. Examine edges leaving *A*, update costs chart
Pick *smallest* edge to an unvisited vertex (*C*)

Unvisited nodes = {B, D, E} *C*

A: 0
B: 1
C: 2
D: 3
E: 4

3. Examine edges leaving *C*, update costs chart
Pick *smallest* edge to an unvisited vertex (*B*)

Unvisited nodes = {D, E} *B* *C*

A: 0
B: 1
C: 2
D: 3
E: 4

4. Examine edges leaving *B*, update costs chart
Pick *smallest* edge to an unvisited vertex (*D*)

Unvisited nodes = {E} *D* *B* *C* *A*

A: 0
B: 1
C: 2
D: 3
E: 4

5. Examine edges leaving *D* (none), update costs chart
Pick *smallest* edge to an unvisited vertex (*E*)

Unvisited nodes = {} *A* *B* *C* *D* *E*

A: 0
B: 1
C: 2
D: 3
E: 4

[illegible]

0/7 **Quiz** – *dynamic programming (reusable recipe)*

1. What is the main idea of dynamic programming?

2. How can we avoid recomputing subproblems?

3. How can we store the results of subproblems?

4. How can we use the results of subproblems to solve the original problem?

5. How can we use the results of subproblems to solve the original problem?

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3	4	4
5	0	1	2	3	4	5

DP table – 2D array of subproblems

subproblems	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	1	1	1	1
2	0	1	2	2	2	2
3	0	1	2	3	3	3
4	0	1	2	3		

Algorithm Design & Analysis — Final Cheat Sheet

Design-Technique Map

Technique	Relationship	Signature algos
Decrease & Conquer (U5)	Solve ONE smaller instance, extend to full	Topological sort, insertion sort, binary search
Divide & Conquer (U6)	Split into SEVERAL subproblems, solve each & combine	Quicksort/Hoare, mergesort
Transform & Conquer (U7)	Transform to simpler instance / other representation / other problem, then solve	Heapify, presorting
Space–Time trade-off (U8)	Precompute/store extra data for fast queries	Counting sort, Horspool, hashing
Dynamic Programming (U9)	Overlapping subproblems solved once, stored in table (bottom-up)	Coin-row, coin-change, 0/1 knapsack, Floyd, Warshall
Greedy (U10)	Sequence of locally-best, unalterable choices	Prim, Kruskal, Dijkstra, Huffman
Backtracking / B&B	Build state-space tree with bounds	Assignment problem

DP vs D&C: both split — but D&C subproblems are **independent**; DP exists **because** subproblems overlap (naive recursion is exponential). **Greedy vs DP:** greedy optimizes the next step & commits (fast, only correct with optimal substructure/greedy-choice); DP considers all choices via a table and optimizes the whole path.

Complexity Reference

Algorithm	Best	Avg	Worst	Space
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$
Topological sort (DFS)	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V)$
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$
Binary-tree traversal	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$
Heapify (bottom-up)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
AVL tree op	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
Horner's rule	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Gaussian elimination	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Counting sort (cmp / distr)	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n)$
Horspool / Boyer–Moore	$\Theta(n/m)$	$\Theta(n)$	$\Theta(nm)$	$\Theta(\alpha)$
Hashing	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Warshall (trans. closure)	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
Floyd (all-pairs SP)	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
0/1 knapsack (DP)	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$
Prim (min-heap+adj)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log E)$	$\Theta(V)$
Kruskal	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(V)$
Dijkstra (min-heap)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(V)$
Huffman coding	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

$\Theta(nW)$ is **pseudo-polynomial** — W is a value, not the input size.

1 · Topological Sort — Source Removal

On a **DAG**, produce a linear order where every edge $u \rightarrow v$ has u before v (like getting dressed: socks before shoes).
Steps: ① find a *source* (no incoming edges) ② output it & delete it + its out-edges ③ repeat until empty.

2 · Array Partition — Hoare

Engine inside quicksort (D&C). Pivot $p=A[l]$; $i=l$, $j=r+1$. Scan **i right** to first element $\geq p$; scan **j left** to first $\leq p$. If $i < j$ swap $A[i], A[j]$ & continue. When $i \geq j$, swap pivot $A[l] \leftrightarrow A[j]$. j is the split point: sp left, $\geq p$ right.

3 · Heap Construction — Bottom-up Heapify

Max-heap = complete binary tree (array) with every parent $\geq$ children; built in $\Theta(n)$ (transform & conquer). For i from last parent $((n/2)-1)$ down to 0: **sift down** — swap with larger child while a child is bigger; stop when $\geq$ both children or a leaf.

4 · MST — Kruskal

MST of weighted connected graph is acyclic with exactly $|V|-1$ edges (cheapest road network). **Steps:** ① sort edges by increasing weight ② add each edge if it does **not** form a cycle ③ stop at $|V|-1$ edges. Closing a cycle means its heaviest edge is never needed.

5 · MST — Prim

Greedy **by vertex**: grow one connected tree from any start, each step adding the cheapest edge from the tree V_T to a new vertex; repeat until all in $(|V|-1)$ edges). **Prim vs Kruskal:** Prim keeps one growing tree; Kruskal takes the globally cheapest edge and may merge several fragments.

6 · Comparison-Counting Sort

Space–time trade-off: count how many elements each one beats. For every pair $i < j$: if $A[i] < A[j]$ then $\text{Count}[j]++$ else $\text{Count}[i]++$. Then $\text{Count}[i] = \#$ elements smaller than $A[i]$ = its final index; place $A[i]$ at $\text{output}[\text{Count}[i]]$. $\Theta(n^2)$.

7 · Coin-Row (DP)

Row of coins $c_1 \dots c_n$; max total, no two adjacent. $F(i) = \max$ value from first i coins. Take $i \rightarrow c_i + F(i-2)$; skip $i \rightarrow F(i-1)$. **$F(i) = \max(F(i-1), c_i + F(i-2))$** , $F(0)=0$, $F(1)=c_1$. Backtrack from the end, jump 2 when a coin is taken.

8 · Money Change (min coins DP)

Fewest coins to make amount m , indexed by **amount**. Ask "what was the last coin c_i ?" $\rightarrow$ before it we had $m-c_i$. **$F(m) = \min \text{ over } c_i \text{ of } F(m-c_i) + 1$** , $F(0)=0$. The $+1$ is the coin placed now; $F(m-c_i)$ solves what's left.

9 · Assignment — Branch & Bound

Assign n people to n jobs, minimize cost (brute force $n!$). ① **Lower bound** = sum of each person's cheapest job ② **branch**: pick next person, child per free job ③ compute each child's bound, explore smallest first, **prune** nodes with bound $\geq$ best complete solution ④ update best when a leaf beats the incumbent.

10 · Dijkstra — single-source SP

Greedy ripples from source (non-negative weights). ① $\text{dist}(\text{src})=0$, others $=\infty$, keep prev pointers ② pop unvisited min-distance vertex (min-PQ), mark visited/final ③ **relax** edges: if $\text{dist}(u) + w(u,v) < \text{dist}(v)$ update $\text{dist}(v)$, $\text{prev}(v)=u$ ④ repeat until all visited.

11 · Huffman Coding

Optimal prefix tree — short codes for frequent chars (greedy). ① leaf per char with frequency in a min-heap ② extract two smallest, make internal node $\text{freq} = \text{sum}$, children become 0/1, reinsert ③ repeat to one root ④ codes = path: left 0, right 1. Rarest chars end deepest.

12 · 0/1 Knapsack (DP)

Items (weight, value), capacity W ; max value, each item once. $F[i][c] = \text{best with first } i \text{ items, capacity } c$. Exclude: $F[i-1][c]$. Include (if $w_i \leq c$): $\text{value}_i + F[i-1][c-w_i]$. **$F[i][c] = \max(F[i-1][c], \text{value}_i + F[i-1][c-w_i])$** . $\Theta(nW)$.

13 · Floyd–Warshall

All-pairs shortest paths, negative edges allowed (no negative cycles). Let paths use intermediate vertices $1 \dots k$: $D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$. **Floyd** = shortest distances; **Warshall** = transitive closure (same recurrence with OR/AND). $\Theta(n^3)$.